SafeVault: Presentation Script & Commands

Automated, Ransomware-Resilient Backup of Institutional Data to Cloud Storage

1. Before you start (20 minutes earlier)

Open **Ubuntu (WSL)** on the laptop and run:

```
cd ~/safevault-lite
sudo service cron start
python3 lite.py schedule --every 5 --install    # type SCHEDULE
python3 lite.py run-backup --select-clean
```

This gives fresh scheduled runs by presentation time. Keep Ubuntu open and the laptop on charge.

Set up the screen:

- **Tab 1:** the SafeVault dashboard (Netlify link). Press **Ctrl+F5**, turn on **Auto-refresh**.
- **Tab 2:** Ubuntu terminal, inside `~/safevault-lite`. Make the text big: **Ctrl + mouse wheel**.
- **Do not open:** AWS access keys, IAM key pages, `.aws` folder, Groq key, Netlify environment variables, billing.

Quick rehearsal check (2 min):

```
python3 lite.py status
```

You should see scheduled runs marked **success**.

2. Running order (8 minutes)

Who	Topic	Time	Screen
Member 1	Problem and architecture	1.5 min	Dashboard: How it works tab
Member 2	Automatic backups	2 min	Terminal + Dashboard
Member 3	Protection and recovery	2 min	Terminal + Dashboard
Member 4	Live ransomware drill + chatbot	2.5 min	Terminal + Dashboard
Everyone	Limitations and close	30 s	Dashboard

3. Member 1: Problem and architecture (1.5 min)

Screen: Dashboard, then click the **How it works** tab.

Say:

> "Institutions like colleges keep attendance, marks and records on computers. Ransomware attacks don't only encrypt these files. They also try to delete the backups, so the institution can't recover without paying."
>
> "Our project, SafeVault, solves this in three ways. One: backups run automatically to Amazon S3. Two: old clean backups are protected so they can't be deleted. Three: we prove we can recover, by restoring and checking every file."
>
> "Here is the flow. A scheduler on Ubuntu runs our Python program. It compresses the files, records a SHA-256 fingerprint of each one, and uploads them to a private S3 bucket in Mumbai, using an IAM user that can **only upload**."
>
> "The bucket has Versioning, Object Lock for 7 days, encryption and HTTPS-only. A separate read-only user does recovery. Results are published to this dashboard. We use synthetic records only, no real student data."

Hand over: "Now Member 2 will show the automation."

4. Member 2: Automatic backups (2 min)

Screen: Terminal.

```
python3 lite.py status
```

Say:

> "This is our scheduler status. cron runs a backup every 5 minutes for the demo, hourly normally. Each run records its start time, end time, result and any error."
>
> "Two runs can never overlap. A lock file stops a second backup if one is still running."

Switch to Dashboard and point to **Last successful backup** and the **Scheduled backup runs** table.

> "Every run publishes its result here automatically. If a backup fails, the failure is shown in red, but the last good backup is never hidden. So the admin always knows the real state."

Optional (if time): run one manual backup live

```
python3 lite.py run-backup
```

> "You can see it uploaded and the dashboard updates within a minute."

Hand over: "Member 3 will show how the backups are protected and recovered."

5. Member 3: Protection and recovery (2 min)

Screen: Terminal.

```
python3 lite.py verify-protection
```

Say:

> "This reads the real settings directly from AWS: Versioning is enabled, Object Lock is on with 7-day Governance retention, encryption is on, and public access is blocked."
>
> "Versioning keeps every older copy. Object Lock means a locked version can't be deleted or overwritten until the date passes."

```
python3 lite.py cloud-restore
```

> "Now we restore from the cloud. We download the exact clean version we selected, not just the latest one, because the latest could already be infected."
>
> "It's restored into a new folder, and every file is compared with fingerprints we saved separately. 5 out of 5 files match, so the backup is usable."

Switch to Dashboard: point to **Latest recovery from S3** and the **Bucket protection** panel.

Hand over: "Member 4 will now attack our own system live."

6. Member 4: Live ransomware drill + chatbot (2.5 min)

Screen: Terminal.

```
python3 lite.py ransomware-drill      # type DRILL
```

Say while it runs:

> "This is a safe simulation, with no real malware. It works on a throwaway copy of our data."
>
> "Step 1: the files are encrypted. Step 2: zero files are readable now."
>
> "Step 3: an attacker steals the laptop's backup key and tries to read, delete and unlock our backups. AWS says **AccessDenied** every time, because that key can only upload."
>
> "Step 4: we recover the clean backup from S3, and all 5 files are verified."

Switch to Dashboard: click **Refresh from S3**. Point to the **Ransomware drill** panel and the green **evidence ring**.

Chatbot: click the chat button in the bottom-right corner and ask:

> *Did the drill pass?*
>
> "Our assistant uses Groq AI through a secure server function. The API key never reaches the browser, and it answers from our real results."

7. Everyone: Close (30 s)

Member 1 says:

> "To be honest about limits: we used synthetic files, the scheduler runs only while the laptop is on, Governance lock can be bypassed by a special admin, and recovery runs on the same laptop. So SafeVault is ransomware-**resilient** under our tested threat model, not ransomware-proof."
>
> "In short: automatic backups, protected versions, least-privilege access, and recovery that is proven, not just claimed. Thank you."

8. If something goes wrong

Problem	What to do
Wi-Fi or AWS is slow	Say "we'll show the recorded result", then show the dashboard (it keeps the last results)
Dashboard shows old data	Click **Refresh from S3**, then Ctrl+F5
Drill or restore fails	Don't panic: `python3 lite.py status` shows earlier passed results. Explain what it checks
Chatbot is slow	Ask again. Without AI it still answers from built-in answers
Nothing works	Click **View demonstration** and clearly say "these are demo values"

9. Likely viva questions

- **Why not just copy files to the cloud?** Anyone with delete access could wipe them. Our uploader can't delete, and Object Lock blocks deletion.
- **Why restore an older version?** The newest upload might already be encrypted. We pin the clean version.
- **IAM deny vs Object Lock?** IAM stops *this key*. Object Lock stops deletion by *anyone* without special bypass.
- **Governance vs Compliance?** Governance can be bypassed by a special permission. Compliance can't be shortened by anyone, even root. We avoided it on a student account.
- **How do you know the restore is correct?** SHA-256 fingerprints are compared with a baseline saved before any damage.
- **Is the dashboard live?** It refreshes every 60 seconds from S3. Every backup or test updates it automatically.
- **Where is the Groq key?** In a Netlify environment variable, used only on the server.
- **Cost?** Small files in S3 Mumbai plus free tiers. Well under a dollar.

10. After the presentation

```
python3 lite.py schedule --remove      # stop automatic backups
```

Or keep it hourly: `python3 lite.py schedule --every 60 --install`